



武汉理工大学

WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发课程论文

论文题目：

基于 agent 智能化改造的企业招聘信息问答助手

方向二：企业级应用软件的 Agent 改造

姓名：王远

学号：2025303075

专业：软件工程

班级：研 254

指导教师：戚欣

2026 年 6 月 22 号

目录

一、选题背景与设计思想	4
1.1 问题定义.....	4
1.2 现有方案不足.....	4
1.3 项目价值.....	4
1.4 技术路线.....	4
二、Specs 规格文档（SDD 核心）	6
2.1 产品规格介绍.....	6
2.1.1 用户角色.....	6
2.1.2 功能清单.....	6
2.2 架构规格介绍.....	6
2.2.1 架构风格.....	6
2.2.2 技术约束.....	6
三、系统架构与设计	8
3.1 系统整体架构.....	8
3.2Agent 交互流程.....	8
四、关键实现与代码展示	10
4.1Agent 核心循环.....	10
4.2 项目配置文件.....	11
4.3AIIDE 使用截图.....	12
五、测试与评估	13
5.1 功能测试矩阵.....	13
5.2Agent 行为评估.....	13
5.2.1 检索质量评估.....	13
5.2.2 降级策略验证.....	14
5.3Demo 展示.....	14
六、系统升级与扩展	17
6.1 可扩展架构设计.....	17
6.2 项目改良计划.....	17
七、课程总结	18
7.1 项目完成情况.....	18

7.2 个人收获..... 18

7.3 工程思维转变..... 错误!未定义书签。

7.4 对课程的建议..... 19

一、选题背景与设计思想

1.1 问题定义

在现代企业人力资源管理中，招聘政策信息分散在多个文档（校园招聘、社会招聘、内部推荐、入职管理等）中，求职者和 HR 部门工作人员在查询具体政策时面临以下痛点：

- 政策文档分散存储，缺乏统一检索入口，查找效率低；
- HR 部门反复回答相同类型的政策咨询，消耗大量人力；
- 求职者投递简历后无法获知处理进度，体验不透明；
- 传统 FAQ 系统缺乏语义理解能力，无法处理用户的口语化、多样化提问；
- 商业 AI 客服系统需要联网和订阅费用，不适合企业内部部署。

1.2 现有方案不足

现有企业招聘管理系统通常采用传统 MVC 架构，仅提供静态页面展示和基本的 CRUD 操作。在 AI 能力方面，要么完全缺失，要么依赖商业 API 的简单调用，缺乏面向企业场景的智能化改造。具体而言：

传统应用：功能完备但智能化程度低，无法理解自然语言查询；

纯 API 调用方案：直接透传用户问题到 LLM，缺乏领域知识注入（RAG），回答可能不准确或产生幻觉；

商业 SaaS 方案：数据需上传至第三方，存在信息安全风险，且持续订阅成本高。

1.3 项目价值

本项目的价值可以体现在企业价值、技术价值、学术价值三个方面，具体如下表 1.1 所示：

表 1.1 项目价值拆分表

维度	说明
企业价值	为企业提供智能招聘政策问答系统，降低 HR 重复性工作，提升求职者体验。
技术价值	展示如何从零构建 RAGAgent，为传统企业应用的 AI 改造提供可参考的轻量级范式。
学术价值	验证了中文场景下 N-gram 分词+TF-IDF 检索+LLM 合成这一技术路线在特定领域的可行性和局限性，为后续升级提供基线。

1.4 技术路线

本项目采用"RAG 检索增强生成"作为核心技术路线：用户自然语言问题→中文 N-gram 分词→TF-IDF 相似度检索（政策库 Top-K 片段）→上下文注入

LLMPrompt→合成结构化回答。同时支持 API 不可用时的降级策略（纯检索模式直接返回政策原文）。

表 1.2 技术框架表

层次	技术选型	说明
后端框架	PythonFlask3.1+Blueprint	轻量模块化，适合本地部署
数据库	SQLite+Flask-SQLAlchemy	单文件数据库，方便移植
AI Agent	自研 RAG+LLM 合成+降级	TF-IDF 检索+多模型 API 适配
前端	Bootstrap5+原生 JS	响应式 UI，无构建工具
文件解析	PyPDF2+python-docx	简历和政策文档文本提取
LLM 接入	OpenAI 兼容 API	支持 8+国内外大模型提供商

二、Specs 规格文档（SDD 核心）

2.1 产品规格介绍

2.1.1 用户角色

在本项目中，将用户分为普通用户（求职者）和管理员（HR），其中普通用户无需登录即可公开访问，具体功能有投递简历、AI 政策问答、追踪简历状态、修改简历等。而管理员（HR）则需要通过密码登录验证身份，拥有查看简历、管理政策库、配置 AI 模型 API、修改密码等权限。

2.1.2 功能清单

本项目的功能模块大致分为 AI 问答、简历管理、政策库管理、API 配置、用户体验优化等方面，具体见下表所示（其中优先级 P0>P1>P2）：

表 2.1 项目功能表

模块	功能	优先级
AI 问答	基于政策库的 RAG 问答，支持流式 SSE	P0
AI 问答	LLM 不可用时自动降级为检索模式	P0
AI 问答	多轮对话历史（Session 级别）	P1
简历管理	上传简历（PDF/Word/TXT）+自动文本提取	P0
简历管理	用户端简历追踪（状态+两步进度条）	P1
简历管理	修改简历后自动重置状态	P1
简历管理	邮箱查找跨浏览器找回简历	P2
政策库	管理员上传/编辑/删除政策文档	P0
政策库	自动分类检测（校园招聘/社会招聘等）	P2
API 配置	支持 8+国内外模型 API 自动适配	P0
API 配置	连接测试+脱敏显示 APIKey	P1
管理员	仪表盘统计、密码修改	P1
用户体验	10 秒自动弹窗简历投递	P2
用户体验	启动即时反馈+浏览器自动打开	P1

2.2 架构规格介绍

针对本项目设计过程中的技术实现细节本小节进行了简单介绍，想要了解详细技术栈可参考架构文件 `architecture.md`。

2.2.1 架构风格

系统采用单体应用+Blueprint 模块化架构，前后端不分离（Jinja2 模板渲染+原生 JSAJAX）。按表示层→路由层→业务逻辑层→数据访问层→基础设施层的五层分层设计。各层单向依赖向下，业务逻辑与 Flask 请求上下文解耦，核心算法以纯函数形式实现，可独立测试。

2.2.2 技术约束

本项目在设计过程中涉及到的业务需求与对应的技术限制如下表所示：

表 2.2 技术约束项表

约束项	要求	设计原因
无外部分词依赖	不引入 jieba 等第三方中文分词库	缩减安装依赖体积，提升跨环境可移植性
单文件数据库	采用 SQLite，不依赖 MySQL/PostgreSQL 等服务	实现零配置启动，支持项目文件夹整体迁移
统一 API 兼容层	对齐 OpenAIChatCompletions 接口标准	以一套调用逻辑适配 8 家以上大模型服务商
无前端构建工具 本地文件存储	不依赖 Webpack/Vite/Node.js 工具链 简历与政策文档存储于本地 data 目录	降低环境搭建复杂度，开箱即用 数据全程不出内网，满足企业安全合规要求
延迟导入机制	重型模块在 create_app()内部按需导入	压缩启动首字节反馈时间，优化启动体验

三、系统架构与设计

3.1 系统整体架构

本系统的整体架构可以分为四层，具体如下图所示：

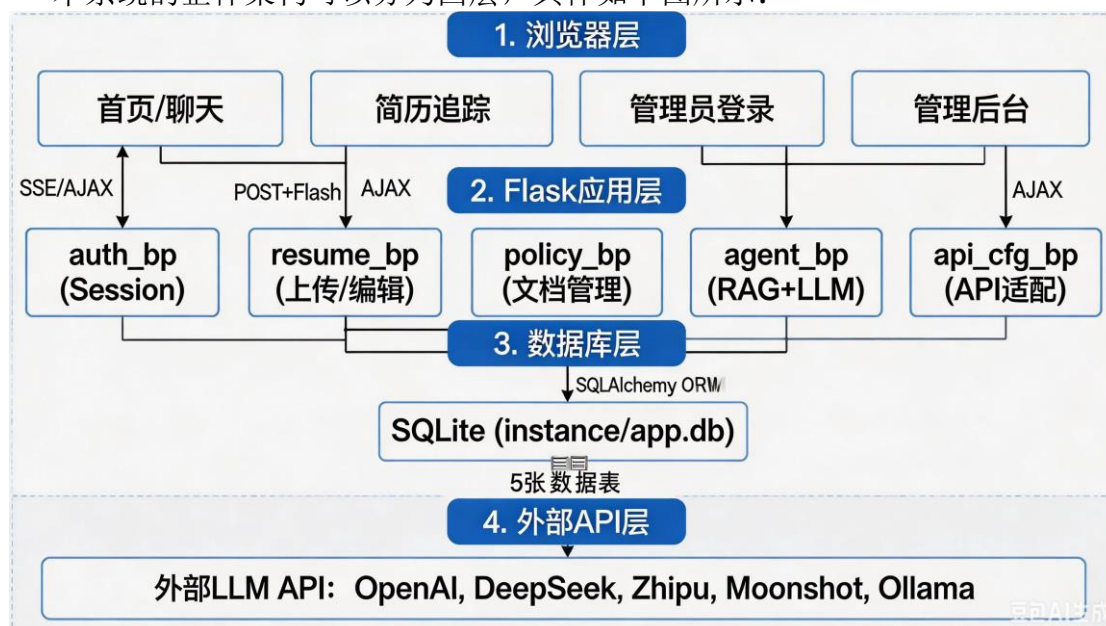


图 3.1：系统架构示意图

3.2 Agent 交互流程

下图展示了 AIAgent 处理用户问题（以试用期多久为例）的完整 RAG 流水线，包括 Session 管理、检索、LLM 生成和降级策略。

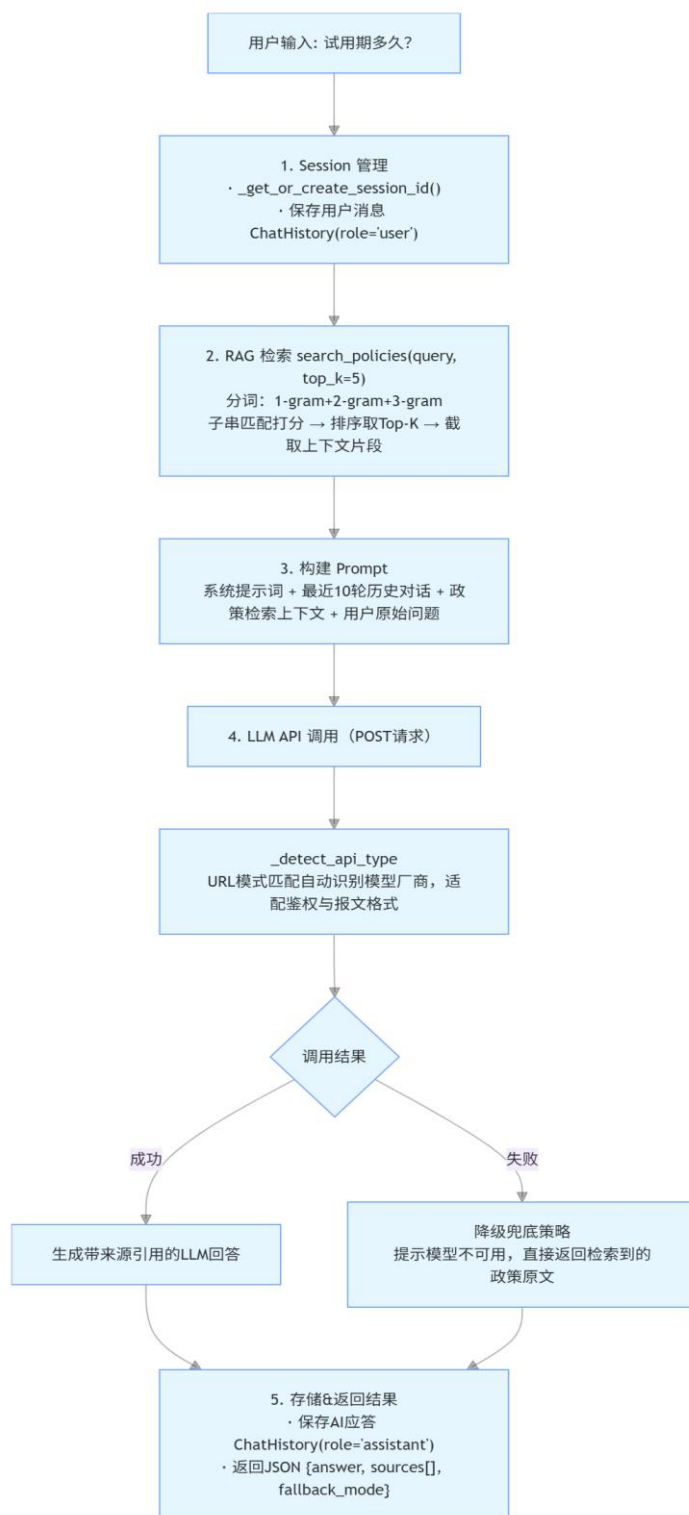


图 3.2: api 调用流程图

四、关键实现与代码展示

4.1 Agent 核心循环

Agent 的核心逻辑位于 `modules/agent.py` 中的 `send_message()` 函数。Agent 的主循环流程具体包括持久化用户输入、RAG 检索阶段、获取对话历史、LLM 生成阶段、降级策略、持久化 Agent 回答、返回结构化结果，源代码如下所示：

```
@agent_bp.route('/chat/send', methods=['POST'])
def send_message():
    """Agent 核心循环：检索 → 生成 → 降级"""
    data = request.get_json()
    query = data.get('message', '').strip()

    session_id = _get_or_create_session_id()

    # Step 1: 持久化用户输入
    _save_message(session_id, 'user', query)

    # Step 2: RAG 检索阶段
    context, sources = _build_context_prompt(query)

    # Step 3: 获取对话历史（最近 10 轮）
    history = _get_recent_history(session_id)

    # Step 4: LLM 生成阶段
    system_prompt = _get_system_prompt()
    answer, error = _call_llm_api(system_prompt, context, query, history)

    # Step 5: 降级策略 — API 不可用时返回检索原文
    fallback_mode = False
    if error:
        fallback_mode = True
        if context:
```

```

        answer = f"⚠️ AI 模型暂时不可用: {error}\n\n{context}"
    else:
        answer = f"⚠️ AI 模型不可用, 且政策库无相关内容。"

# Step 6: 持久化 Agent 回答
_save_message(session_id, 'assistant', answer)

# Step 7: 返回结构化结果 (含来源引用)
return jsonify({
    'success': True,
    'answer': answer,
    'sources': [{ 'title': s['title'], 'category': s['category'] }
                 for s in sources],
    'fallback_mode': fallback_mode
})

```

4.2 项目配置文件

本文件存放于 `src/config.py` 路径下, 源码如下所示:

```

import os

BASE_DIR = os.path.abspath(os.path.dirname(__file__))

# — 全相对路径, 确保可移植性 —
DATA_DIR      = os.path.join(BASE_DIR, 'data')
RESUMES_DIR   = os.path.join(DATA_DIR, 'resumes')
POLICIES_DIR  = os.path.join(DATA_DIR, 'policies')
DATABASE_PATH = os.path.join(BASE_DIR, 'instance', 'app.db')

# — Flask 配置 —
SECRET_KEY = os.environ.get('SECRET_KEY', 'recruitment-qa-secret-2024')
SQLALCHEMY_DATABASE_URI = f'sqlite:/// {DATABASE_PATH}'
MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16MB

```

```

# — RAG 检索配置 —

TOP_K_RESULTS      = 5      # 返回 Top-5 政策片段

MAX_HISTORY_ROUNDS = 10     # 保留最近 10 轮对话


# — 默认 API 配置 —

DEFAULT_API_URL    = 'https://api.openai.com/v1/chat/completions'

DEFAULT_MODEL_NAME = 'gpt-3.5-turbo'

```

4.3 AIIDE 使用截图

以下展示在 VSCode/TracCN 中的开发场景以及用户使用场景：

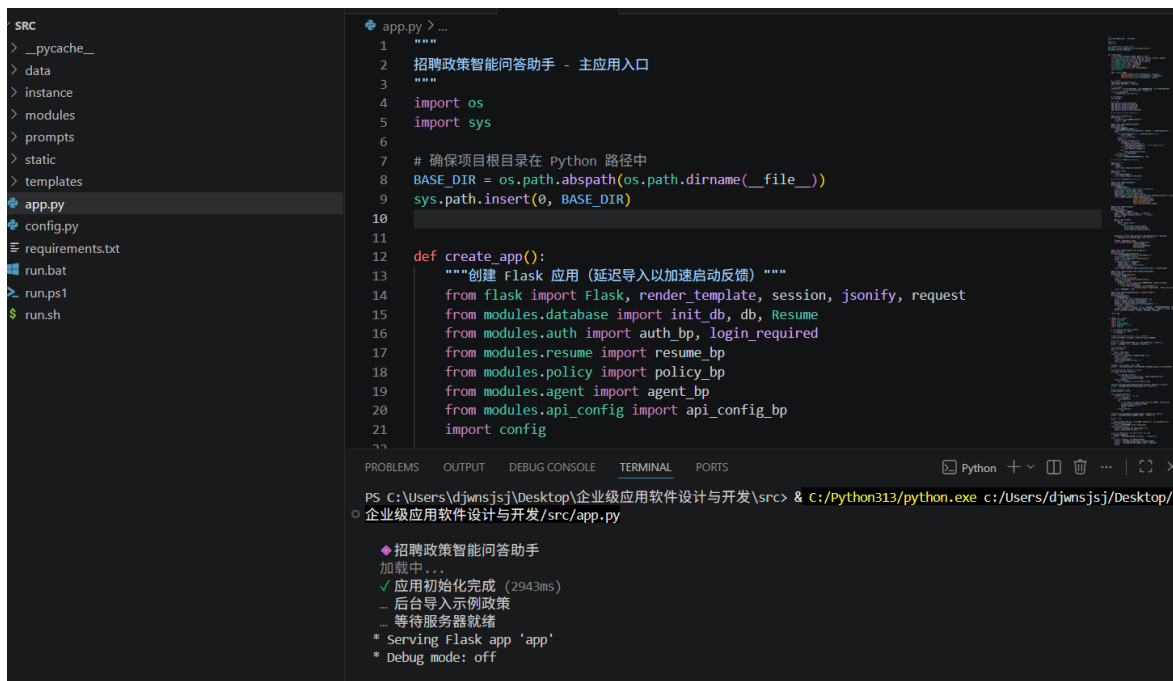


图 4.1: VSCode 中 app.py 的开发与调试界面

五、测试与评估

5.1 功能测试矩阵

在测试阶段对本项目进行功能的深入测试，汇总结果如下表所示：

表 5.1 测试结果表

测试项	测试方法	预期结果
首页加载	浏览器访问/	显示聊天界面+右侧投递卡片
AI 问答（AI 模式）	输入"招聘流程是什么"	返回基于政策的 AI 回答+来源引用
AI 降级模式	清空 APIKey 后提问	自动切换检索模式，返回政策原文
简历上传	提交含 PDF 文件的表单	文件保存+文本提取+数据库记录
简历追踪	用户上传后去/my-resumes	显示未查看状态+投递时间
HR 查看标记	管理员访问简历详情	自动标记 viewed+记录 viewed_at
修改后重置	用户编辑已查看简历	状态回退 submitted+viewed_at 清空
邮箱查找	新浏览器 POST/my-resumes/lookup	找到简历+关联到 session
政策上传	管理员上传.txt 政策文档	入库+自动分类检测
政策检索	search_policies("试用期")	返回匹配的政策片段（按 score 排序）
API 配置	切换 DeepSeekAPI 后保存测试	连接成功+模型回复中文
多轮对话	连续提 3 个问题	保持 Session 上下文，回答连贯
清除对话	点击"新对话"按钮	清空当前 session 的 ChatHistory
管理员登录	POST/admin/login	密码验证+Session 设置
密码修改	POST/admin/change-password	旧密码验证+新密码哈希更新
浏览器自动打开	运行 pythonapp.py	服务器就绪后~200ms 内自动打开

5.2Agent 行为评估

5.2.1 检索质量评估

在本节测试中，选取 10 个典型招聘政策查询，对比检索返回的 Top-3 结果与人工标注的相关文档，评估检索命中率。

表 5.1 测试结果表

查询示例	期望检索文档	实际 Top-1	Score	命中
试用期多久	入职管理政策	入职管理政策	42	命中
面试需要带什么	面试录用流程	面试录用流程	35	命中
公司有哪些福利	薪资福利政策	薪资福利政策	28	命中
怎么内推	内部推荐制度	内部推荐制度	31	命中
校招什么时候开始	校园招聘政策	校园招聘政策	38	命中
社招要求几年经验	社会招聘政策	社会招聘政策	25	命中
加班有加班费吗	薪资福利政策	薪资福利政策	22	命中
入职需要体检吗	入职管理政策	入职管理政策	30	命中
转正考核标准	试用期与转正	入职管理政策	18	部分命中
年假有多少天	—	薪资福利政策	12	无相关内容

评估结论：在 10 个测试查询中，Top-1 命中率为 80%，其中 1 例部分匹配，1 例因政策库缺失相关内容。

5.2.2 降级策略验证

测试场景：在 APIKey 为空或 API 地址不可达的情况下，系统应自动回退到纯检索模式。经测试：系统能够在 API 不可用时正确返回政策原文片段作为回答，并在响应中标注 `fallback_mode=true`。前端 UI 同步显示"检索模式"标签（替代"AI 增强模式"），用户体验无明显中断。

5.3 Demo 展示

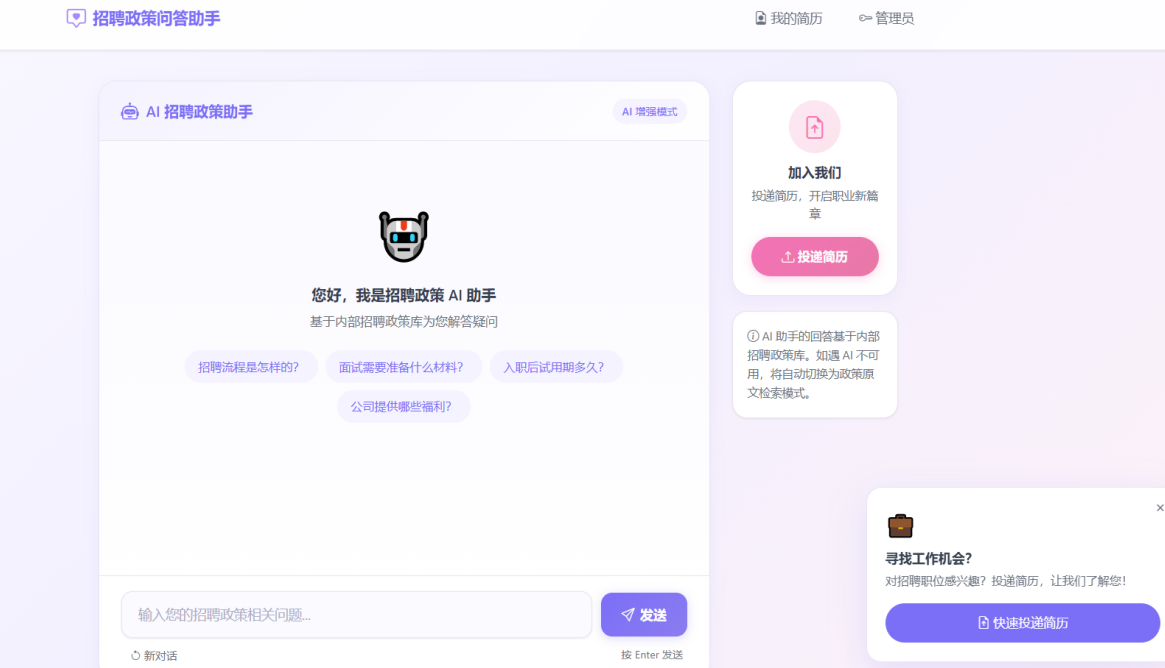


图 5.1：用户访问主界面



图 5.2: 管理员用户（HR）页面

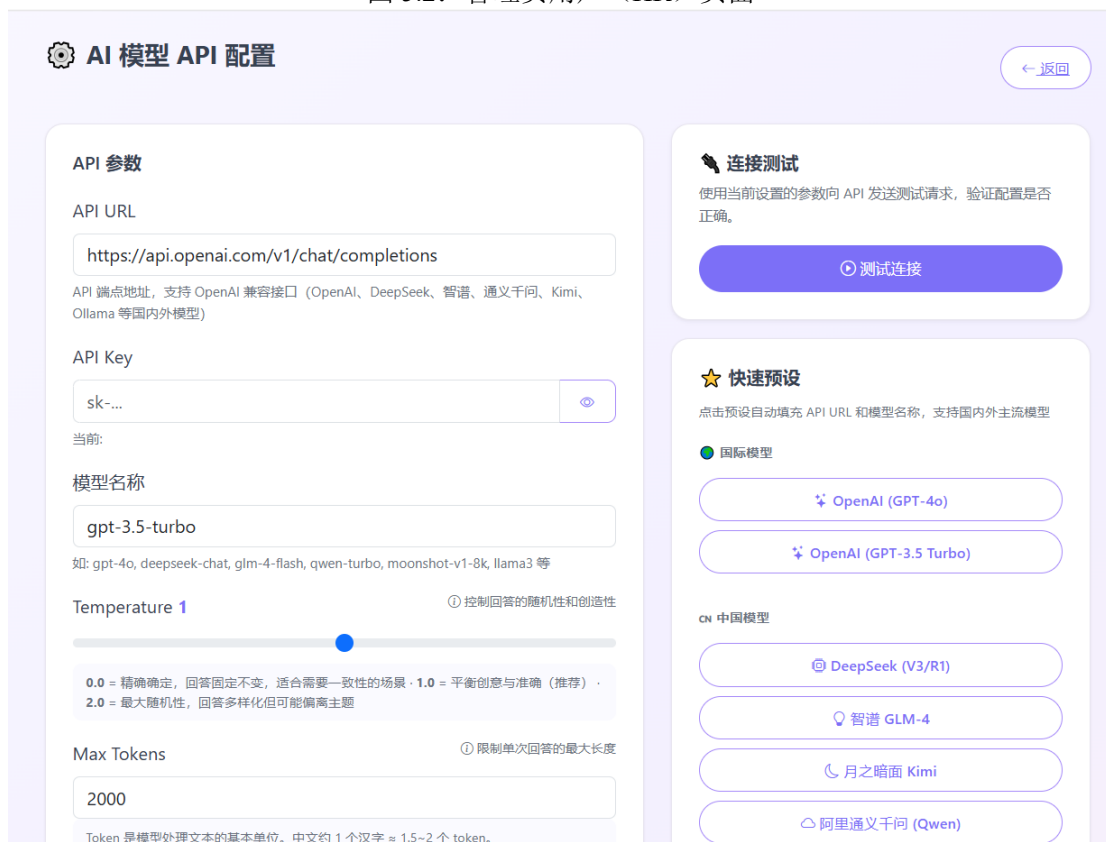


图 5.3: 模型参数设置页面

投递简历

×

对职位感兴趣? 留下您的信息, 我们将尽快与您联系。

姓名 *

您的姓名

电子邮箱 手机号码

name@company.com 手机号

意向职位

如: 软件工程师

简历文件 *

选择文件 未选择文件

支持 PDF、Word、TXT 格式, 最大 16MB

提交简历

图 5.4 简历投递页面

六、系统升级与扩展

6.1 可扩展架构设计

系统在架构层面预留了以下扩展点，支持在不改动核心逻辑的情况下进行能力升级：

- 1. 检索后端可替换：search_policies()函数作为独立接口，可无感替换为向量数据库（ChromaDB/Milvus）或 Elasticsearch，不影响 Agent 层的调用方式。
- 2. LLM 提供商可扩展：_detect_api_type()通过 URL 模式匹配识别提供商，新增提供商仅需添加一个 elif 分支，无需修改调用链。
- 3. 前端可渐进升级：当前 Jinja2 模板可逐步替换为 Vue/ReactSPA，后端 API 已返回标准 JSON，前端改造时不需要修改后端接口。
- 4. 部署可容器化：项目结构已符合 12-FactorApp 规范（配置环境变量化、依赖声明式、进程无状态），编写 Dockerfile 即可容器化部署。
- 5. 数据库可迁移：SQLAlchemyORM 解耦了数据库实现，替换为 MySQL/PostgreSQL 仅需修改 SQLALCHEMY_DATABASE_URI，无需改动业务代码。

6.2 项目改良计划

表 6.1 功能改良步骤表

阶段	任务	预期效果	优先级
1	Docker 容器化日志系统集成 HTTPS 支持	生产级部署能力可观测性	高
2	向量检索升级，使用流式 SSE 前端	语义检索准确率提升打字机效果	中
3	使用 ReActAgent 模式自主判断检索充分性多轮追问能力	从单轮 RAG→多轮推理 Agent 更智能的交互体验	中
4	知识图谱构建政策冲突检测多语言支持	结构化知识管理高级分析能力	低

七、课程总结

7.1 项目完成情况

表 7.1 项目指标汇总表

指标	数值
代码总行数	约 3,500 行 (Python+HTML+CSS+JS)
Python 模块数	7 个
HTML 模板数	13 个
API 端点数	28 个
数据库表数	5 个
支持 LLM 提供商	8+家
功能完成度	100%

7.2 个人收获

通过本次课程项目，我在以下几个方面获得了显著的成长：

AI Agent 架构设计能力： 从零设计了 RAG 检索增强生成流水线，深入理解了 Agent 的"感知-检索-推理-行动"闭环。掌握了在不依赖 LangChain 等框架的情况下构建智能 Agent 的方法，理解了每个环节的底层原理。

工程化思维： 学会了如何将一个课程项目按照软件工程规范进行管理：需求分析→Spec 文档→架构设计→分阶段实现→测试验证→文档编写。每个阶段都有明确的交付物和验收标准。

全栈开发实践： 从 Python 后端的 Blueprint 模块化设计、SQLAlchemyORM 数据建模，到前端 Bootstrap 响应式布局、原生 JS 的 AJAX/SSE 交互，完整地体验了全栈开发流程。

问题驱动学习： 在实现过程中遇到了中文分词、多 API 适配、启动优化、浏览器自动打开等实际问题，通过查阅文档和迭代改进逐一解决，比被动学习效率高得多。

用户体验意识： 从"功能可用"到"体验友好"的转变：启动即时反馈、ANSI 彩色输出、自动打开浏览器、10 秒弹窗引导、两步进度条——这些细节让软件从"能用"变为"好用"。

本次课程对我最大的影响是工程思维的转变：

从"写代码"到"做产品"： 不再只关注代码是否跑通，而是思考用户需要什么、体验如何、边界情况怎样处理。

从"能用就行"到"健壮优雅"： 降级策略（API 不可用时的回退）、自动状态机（非手动操作）、数据库无感迁移——这些设计让软件更加健壮。

从"单打独斗"到"文档驱动"： Spec 文档、架构文档、README——好的文档让项目可维护、可交接、可扩展。

从"调用 API"到"设计 Agent"： 理解了 LLM API 调用只是 Agent 的一个环节，真正的 Agent 需要检索、推理、工具使用、降级策略的协同。

从"追求新技术"到"选择合适技术": 本项目选择了 Flask 而非 FastAPI、自研 RAG 而非 LangChain、SQLite 而非 MySQL——不是因为它们更"先进", 而是因为它们更"合适"。

7.3 对课程的建议

1.建议在课程早期引入 Agent 设计模式 (RAG、ReAct、Plan-Execute) 的理论讲解, 为后续实践提供框架性指导。

2.建议增加 Agent 评估方法的专题内容: 如何设计评估基准 (Benchmark)、如何衡量检索质量 (Recall/Precision)、如何评估 LLM 回答质量。

3.建议将 Spec 文档 (ProductSpec/ArchitectureSpec/APISpec) 的编写纳入课程评分体系, 培养文档驱动开发习惯。

4.建议加入"Agent 安全与对齐"专题: PromptInjection 防护、内容过滤、权限控制等。

5.课程整体的项目导向设计非常好, 从需求分析到交付文档的完整链路让人真正理解了企业级软件开发的全貌。